

중고거래 물건, 얼마에 팔고 배송비는 누가 낼까?

Mercari 중고거래 데이터로 살펴본 인공지능(AI) 가격·배송비 예측 이야기

1차 분석 보고서 (쉽게 풀어쓴 버전 · 전체 데이터 재실행판)

작성자: 정찬성

작성일: 2026년 8월 27일

대상 노트북: 11_캐글_mercari_gradientboost_rmsle_aic_bic.ipynb (전체 1,482,535 건 재실행판)

1. 들어가며

1.1 왜 이런 분석을 하게 됐나

중고 물건을 팔아본 분이라면 다들 한 번쯤 이런 고민을 해보셨을 겁니다. “이 옷, 얼마를 받아야 적당할까?”, “배송비는 내가 내는 게 맞을까, 아니면 사는 사람이 내는 게 맞을까?” 하는 고민 말입니다.

미국·일본의 대표적인 중고거래 앱인 Mercari(메루카리)에서도 판매자들이 똑같은 고민을 합니다. 그래서 이번 분석은 “상품 이름, 카테고리, 브랜드, 상태, 설명글” 같은 정보만 있으면 인공지능이 “이 정도 가격이 적당해요”, “배송비는 보통 이렇게 부담해요” 라고 알려줄 수 있는지 확인해 보는 것을 목표로 했습니다.

이번 보고서는 148 만 건이 넘는 실제 판매 기록 전체를 가지고 분석한 결과입니다. 예전 1 차 분석(2026-08-26)에서는 시간을 아끼려고 2 만 건만 무작위로 뽑아서 봤었는데, 이번에는 표본을 뽑지 않고 데이터 전체(1,482,535 건)를 그대로 사용해 다시 확인했습니다.

또한 “얼마나 잘 맞았는가(RMSLE)”라는 점수 하나만으로는 “이 모델이 적당히 똑똑한 건지, 아니면 문제만 억지로 외운 건 아닌지” 까지는 알 수 없습니다. 그래서 AIC·BIC 라는 보조 점수도 함께 계산해서, 모델이 너무 복잡해지지 않았는지 곁들여 살펴봤습니다.

1.2 이번 보고서가 확인하려는 것

이번 보고서는 크게 두 가지를 확인합니다.

- 가격을 맞추는 모델과 배송비 부담자를 맞추는 모델, 이 두 가지를 전체 데이터로 다시 학습·검증해서, 2 만 건 표본으로 봤던 예전 결과와 얼마나 비슷한지 확인합니다.
- 나무(트리) 개수를 늘려가며 AIC·BIC 로 “어디까지 늘리는 게 적당한가” 를 확인하는 실험도 진행했는데, 전체 데이터로는 한 번 학습하는 데만 4 시간 가까이 걸려서 이번 보고서 시점까지 모든 경우를 다 마치는 못했습니다. 미완료 상태 그대로 솔직하게 적었습니다.

2. 어떤 데이터를 썼나

2.1 데이터는 어디서 왔나

데이터 분석 대회 사이트인 Kaggle 에 올라와 있는 “Mercari Price Suggestion Challenge”의 판매 기록 파일(mercari_train.tsv)을 그대로 사용했습니다. 표본을 따로 뽑지 않고 전체 148 만 2,535 건을 다 사용했습니다.

항목	값
원본 전체 건수	1,482,535 건
이번 분석에 쓴 건수	1,482,535 건 (표본 뽑지 않고 전체 사용)
전체 항목(컬럼) 수	8 개 (식별번호 1 + 가격 1 + 배송비부담 1 + 나머지 정보 5)
맞혀야 할 것 ①	price — 실제 판매 가격 (숫자로 예측)
맞혀야 할 것 ②	shipping — 배송비를 누가 내는지 (구매자/판매자, 둘 중 하나로 예측)
결과 확인용으로 따로 떼 건수	296,507 건 (전체의 20%)

※ 예전1 차 분석(2026-08-26)에서는 2 만 건만 뽑아서 봤습니다. 이번 문서는 표본을 없애고 전체 데이터로 다시 확인한 결과이며, 두 결과의 차이는 5.1 절에서 비교했습니다.

2.2 이 데이터는 어떤 모습인가

이번 데이터는 숫자만 있는 게 아니라 글(상품 이름, 상품 설명), 브랜드 이름, 여러 단계로 나뉜 카테고리, 순서가 있는 상품 상태 등급처럼 성격이 다른 정보들이 뒤섞여 있습니다. 표본을 전체 데이터로 넓히면서, 상품 이름·브랜드에 등장하는 단어의 종류도 훨씬 다양해졌습니다(자세한 내용은 2.7 절).

2.3 각 항목(컬럼)이 뜻하는 것

데이터 안의 8 개 항목은 모두 이름만 봐도 무슨 뜻인지 알 수 있는 것들입니다.

항목 이름	무슨 뜻인가
train_id	물건 하나하나에 매겨진 고유 번호
name	상품 이름 (짧은 문장)
item_condition_id	상품 상태 등급 (1~5, 숫자가 클수록 상태가 나쁨)
category_name	카테고리 (“대분류/중분류/소분류” 형식)
brand_name	브랜드 이름 (적지 않은 경우도 많음)
price	실제 판매 가격 — 이번에 맞히고 싶은 값 ①
shipping	배송비 부담 주체(0/1) — 이번에 맞히고 싶은 값 ②
item_description	상품 상세 설명 (대개 긴 문장)

2.4 브랜드를 안 적은 경우와 카테고리 처리

brand_name(브랜드) 항목은 개인 판매자가 브랜드를 적지 않은 경우가 전체의 약 43%나 될 만큼 비어 있는 경우가 많습니다. 그런데 “브랜드를 적었는가, 안 적었는가” 그 자체가 가격을 가늠하는 중요한 단서였습니다(3.2 절 피쳐 중요도 1 위). 그래서 이 항목을 버리지 않고, 비어 있는 칸을 ‘브랜드없음’ 이라는 값으로 채워서 그대로 모델에 넣었습니다. 카테고리도 마찬가지로 빈 칸을 ‘브랜드없음’ 과 같은 방식으로 채운 뒤, “대분류·중분류·소분류” 3 개 항목으로 나눠서 사용했습니다. 전체 데이터로 확인해 보니 이 6 개 항목 모두 빈 칸이 남김없이 채워졌습니다.

2.7 글자를 숫자로 바꾸는 과정에서 늘어난 항목 수

컴퓨터는 “나이키” , “귀엽다” 같은 글자를 그대로 이해하지 못합니다. 그래서 상품 이름과 설명에 등장하는 단어 하나하나를 “이 단어가 등장했는지, 몇 번 등장했는지” 를 나타내는 숫자로 바꿔주는 과정을 거칩니다. 이 과정을 거치면 항목(피쳐) 개수가 크게 늘어나는데, 표본(2 만 건)에서는 약 6 만 3 천 개였던 항목 수가 전체 데이터에서는 약 16 만 2 천 개까지 늘어났습니다. 특히 상품 이름에서 나오는 단어 종류가 표본보다 9.5 배 넘게 늘어난 것이 가장 큰 이유입니다.

구분	표본(2 만 건) 기준	전체(148 만여 건) 기준
상품 설명에서 뽑은 단어 항목	50,000 개 (설정된 최대치)	50,000 개 (동일)
상품 이름에서 뽑은 단어 항목	11,110 개	105,757 개
브랜드·상태·카테고리 항목 합계	1,735 개	5,811 개

구분	표본(2 만 건) 기준	전체(148 만여 건) 기준
전체 합계 (shipping 제외)	62,845 개	161,568 개

2.8 고유번호(ID) 항목

train_id 는 물건마다 붙는 단순한 순번이라 가격이나 배송비를 예측하는 데는 아무 도움이 되지 않습니다. 그래서 처음부터 모델에 넣는 항목 목록에서 제외했습니다.

2.9 맞히려는 값들은 어떤 분포를 보이냐

가격(price)은 저렴한 물건이 대부분이고 아주 비싼 물건은 소수인, 한쪽으로 쏠린 분포를 보입니다. 배송비(shipping)는 전체 데이터 기준으로 구매자가 부담하는 경우가 55.3%, 판매자가 부담하는 경우가 44.7%로 나타났는데, 이는 예전 2 만 건 표본(55.7% : 44.3%)과 거의 똑같은 비율입니다. 표본을 뽑았던 방식이 전체 데이터의 특징을 왜곡하지 않았다는 뜻입니다.

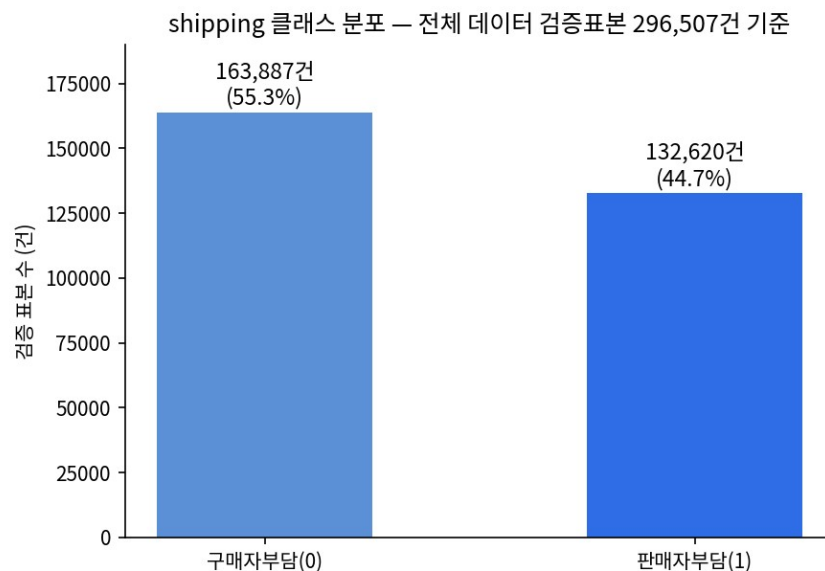


그림 2-1. 배송비 부담 주체 분포 (검증용으로 떼어 둔 296,507 건 기준)

3. 데이터를 직접 들여다보기

3.1 숫자로 바꾼 데이터의 크기 확인

이번 데이터는 글과 범주가 섞인 데이터라서, 흔히 말하는 평균·표준편차보다는 “각 정보 조각이 숫자로 바뀐 뒤 크기가 얼마나 되는가”를 확인하는 방식으로 점검했습니다.

정보 조각	숫자로 바꾼 방식	결과 크기 (건수 × 항목 수)
상품 이름	단어 등장 횟수 세기	1,482,535 × 105,757
상품 설명	단어 중요도 점수화(1~3 개 단어 묶음)	1,482,535 × 50,000
브랜드·상태·카테고리 5 종	있음/없음으로 펼치기	1,482,535 × 5,811
최종 합친 표(shipping 제외)	위 조각들을 옆으로 이어붙임	1,482,535 × 161,568

3.2 가격을 맞힐 때 무엇을 가장 많이 참고했나

모델이 실제로 어떤 정보를 가장 많이 참고해서 가격을 맞췄는지 확인해 봤습니다. 전체 데이터로 다시 확인해도 표본 때와 큰 흐름은 비슷했고, 9~10 위에 ‘Beauty(미용)’ 카테고리과 ‘Lululemon’ 브랜드가 새로 등장해 표본에서는 보이지 않던 세부 패턴이 추가로 드러났습니다.

순위	어떤 정보인가	구체적인 값
1	브랜드를 적었는지 여부	브랜드없음
2	상품 이름 속 단어	“lularoe” (인기 의류 브랜드명)
3	중분류 카테고리	Shoes(신발)
4	중분류 카테고리	Women's Handbags(여성 가방)
5	상품 설명 속 단어	“box”(박스 포함)
6	소분류 카테고리	Cell Phones & Smartphones(휴대폰)
7	상품 설명 속 단어	“authentic”(정품)
8	상품 이름 속 단어	“bundle”(묶음판매)
9	대분류 카테고리	Beauty(미용)
10	브랜드	Lululemon

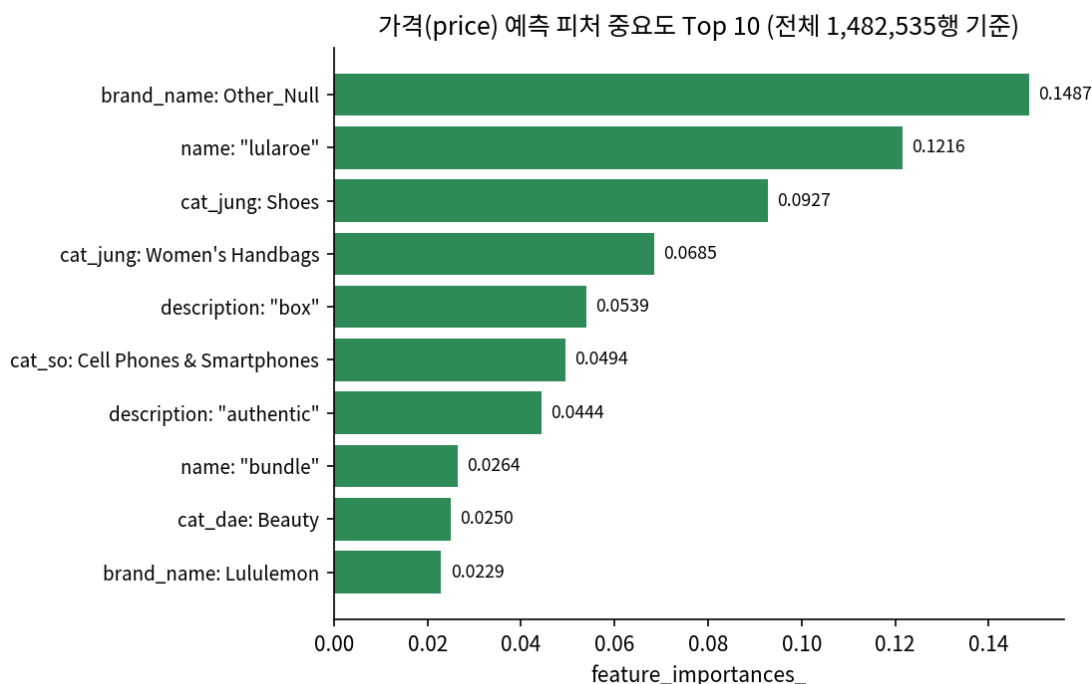


그림 3-1. 가격 예측에 가장 큰 영향을 준 정보 Top 10 (전체 1,482,535 건 기준)

브랜드를 적지 않은 경우와 상품 이름에 “lularoe”가 들어간 경우가 여전히 1·2 위를 차지했고, 신발·여성가방 같은 카테고리가 가격을 크게 가른다는 결론도 표본 때와 같았습니다. 전체 데이터에서만 새로 보인 미용·룰루레몬 항목은, 표본(2 만 건)에는 해당 사례가 충분히 많지 않아 눈에 띄지 않았던 것으로 보입니다. 데이터가 많아지니 자주 나오지 않는 패턴까지 더 안정적으로 잡아낸 셈입니다.

3.3 특이한 값(이상치)은 어떻게 처리했나

브랜드·카테고리·상품설명이 비어 있는 경우는 표본과 전체 데이터 모두 동일하게 ‘브랜드없음’ 같은 값으로 채워 넣었습니다. 가격이 한쪽으로 쏠려 있는 것도 넓게 보면 특이값 문제로 볼 수 있는데, 이는 3.6 절에서 설명하는 로그 변환으로 완화했습니다.

3.4 배송비 예측, 어느 쪽을 더 잘 맞히고 어느 쪽을 더 못 맞히나

앞서 본 평균 점수만으로는 구매자부담·판매자부담 중 어느 쪽을 더 잘 맞히는지 보이지 않습니다. 그래서 두 그룹을 따로 떼어서 다시 확인했습니다.

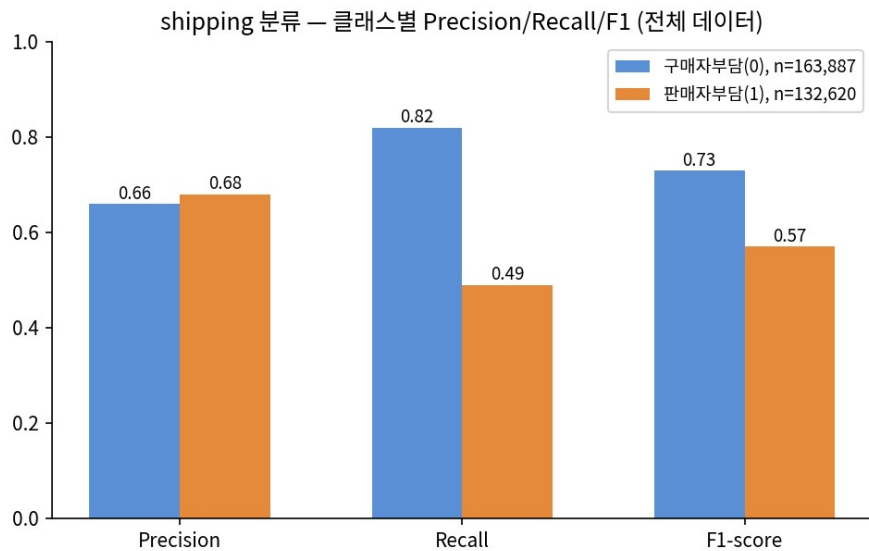


그림 3-2. 배송비 부담 주체 예측 — 구매자부담(0) vs 판매자부담(1) 세부 점수 비교

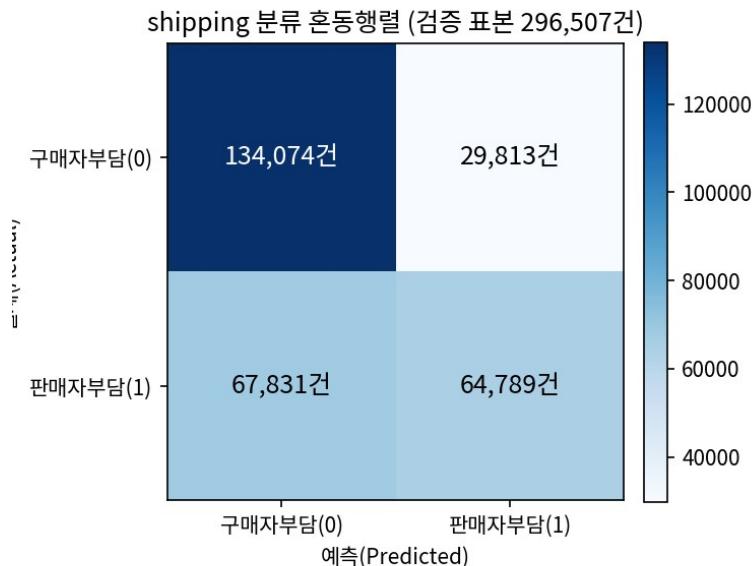


그림 3-3. 배송비 부담 주체 예측 결과표 (검증용 296,507 건 기준)

구매자가 부담하는 경우는 82%를 제대로 맞혔지만, 판매자가 부담하는 경우는 49%밖에 맞히지 못했습니다. 표에서도 “실제로는 판매자가 냈는데 구매자가 낸 것으로 잘못 예측” 한 경우가 67,831

건으로, 그 반대(29,813 건)보다 두 배 넘게 많습니다. 이 현상은 2 만 건 표본에서도 똑같이 나타났던 것이라, 표본이라서 우연히 그런 게 아니라 데이터 자체에 있는 특징이라고 볼 수 있습니다.

3.5 모델에 넣을 최종 정보 확정

2.7 절에서 만든 7 개 조각(브랜드, 카테고리 3 종, 상품 이름, 상품 설명, 상태등급)을 옆으로 이어붙여 최종 표(1,482,535 행 × 161,568 열)를 완성했습니다. 배송비(shipping)는 Part B(배송비 예측)에서 맞춰야 할 정답이므로, 정답을 미리 알려주는 샘플 되지 않도록 가격을 맞추는 모델에서도 일부러 빼고 학습시켰습니다.

3.6 가격에 로그 변환을 적용한 이유

가격은 저렴한 물건이 대부분이고 아주 비싼 물건은 소수라, 그대로 학습시키면 소수의 비싼 물건에 모델이 너무 민감하게 반응할 수 있습니다. 그래서 가격 대신 $\log_{10}(\text{price})$ 라는, 가격에 로그를 씌운 값을 학습 목표로 사용했습니다. 이렇게 하면 “1 만 원짜리를 9 천 원으로 예측” 한 것과 “100 만 원짜리를 99 만 원으로 예측” 한 것을 “둘 다 10% 정도 차이” 로 비슷하게 다룰 수 있어, 값이 작은 물건과 큰 물건을 더 공평하게 평가할 수 있습니다. 이번에 쓰는 평가 점수(RMSLE)도 로그를 씌운 값끼리의 차이를 재는 방식이라, 학습 목표와 평가 기준이 같은 기준 위에서 일관되게 맞춰집니다.

4. 어떻게 분석했나

4.1 전체 진행 순서

데이터를 다루는 순서는 2 만 건 표본 분석 때와 완전히 똑같습니다. 표본을 뽑는 단계만 없었을 뿐, 나머지 처리 방식은 그대로 유지했습니다.

단계	내용
1	카테고리를 “/” 기준으로 대분류·중분류·소분류 3 개 항목으로 나눔
2	브랜드·카테고리·상품설명 빈 칸을 ‘브랜드없음’ 같은 값으로 채움
3	상품 이름 → 단어 등장 횟수, 상품 설명 → 단어 중요도 점수로 숫자화
4	브랜드·상태등급·카테고리 3 종 → 있음/없음으로 펼침
5	7 개 조각을 이어붙여 최종 표(1,482,535 × 161,568) 완성, shipping 은 제외

4.2 이번에 쓴 모델 — Gradient Boosting

Gradient Boosting 은 나무(트리) 여러 개를 순서대로 심어가면서, 앞선 나무가 놓친 부분을 다음 나무가 조금씩 보완해 나가는 방식의 인공지능 기법입니다. 나무 한 그루 한 그루는 그리 복잡하지 않지만, 이런 나무를 수십~수백 개 순서대로 쌓으면 꽤 정교한 예측이 가능해집니다.

이번 분석에서는 같은 방식을 두 가지 용도로 썼습니다. 하나는 가격(연속된 숫자)을 맞추는 용도(GradientBoostingRegressor), 다른 하나는 배송비 부담 주체(구매자나 판매자나, 둘 중 하나)를 맞추는 용도(GradientBoostingClassifier)입니다. 두 모델은 “나무를 순서대로 쌓아 오차를 보완한다” 는 기본 원리는 같지만, 무엇을 오차로 보고 배우는지가 다릅니다.

설정값	이번에 쓴 값	쉬운 설명
나무 개수($n_{\text{estimators}}$)	100	순서대로 심을 나무의 개수

설정값	이번에 쓴 값	쉬운 설명
나무 깊이(max_depth)	3	나무 한 그루가 가지를 뻗는 최대 단계 — 얇은 나무를 여러 개 쌓는 방식
학습률(learning_rate)	0.05	나무 한 그루가 최종 결과에 반영되는 비율(조금씩 반영)
시드값(random_state)	156	다시 실행해도 같은 결과가 나오게 고정하는 값
가격 모델의 기준(loss)	squared_error(기본값)	예측과 실제 가격의 차이(제곱)를 줄이는 방향으로 학습
배송비 모델의 기준(loss)	log_loss(기본값)	“구매자일 확률/판매자일 확률”의 오차를 줄이는 방향으로 학습

4.3 모델이 너무 복잡해지지 않았는지 확인 — AIC·BIC

RMSLE 같은 점수는 “얼마나 잘 맞았는가”만 알려줄 뿐, “그 정확도를 얻으려고 모델이 얼마나 복잡해졌는가”는 알려주지 않습니다. AIC·BIC는 여기에 “너무 복잡한 거 아니야?”하고 확인하는 벌점을 더해 계산하는 점수입니다. 나무를 많이 쓸수록 정확도는 조금씩 좋아지는 경향이 있지만, 나무 개수가 늘어나는 만큼 벌점도 함께 늘어나기 때문에, 어느 지점부터는 “더 늘려봐야 실속이 없다”는 신호를 AIC·BIC가 보여줄 수 있습니다.

다만 Gradient Boosting은 원래 선형회귀 같은 통계 모델을 위해 만들어진 AIC·BIC 계산법에 정확히 들어맞는 모델이 아닙니다. “모델이 몇 개의 기준(파라미터)을 배웠는가”를 나무들의 최종 가지 끝(리프) 개수로 근사해서 계산했다는 점을 감안하고 참고용으로만 봐야 합니다.

항목	표본(2만 건) 기준	전체(148만여 건) 기준
검증용으로 댄 건수	4,000 건	296,507 건
오차 제곱의 합	약 1,720	약 126,636
기준(파라미터) 근사값	795	801
AIC	-1,785.5	-250,652.0
BIC	3,218.2	-242,161.5

※ 나무 개수(20/50/100/200)를 바꿔가며 비교하는 실험은 전체 데이터 기준 한 번 학습에 약 3.9 시간이 걸려, 이번 보고서 시점까지 모든 경우를 다 끝내지 못했습니다. 준비되는 대로 다음 보고서에 반영할 예정이며, 아래 5.1 절의 해석은 나무 100 개 조건의 결과만을 근거로 합니다.

표본 때와 비교하면 AIC·BIC 숫자 자체가 크게 달라졌는데, 이는 모델이 갑자기 좋아지거나 나빠져서가 아니라 계산식 자체가 검증 건수(n)가 늘어날수록 커지도록 되어 있기 때문입니다. 그래서 건수가 다른 두 결과를 숫자 그대로 맞대어 비교하기보다는, 이번 계산 과정을 투명하게 남겨두는 기록으로 활용하는 것이 맞습니다.

4.4 학습에 걸린 시간

항목	표본(2만 건)	전체(148만여 건)
가격 모델 학습 시간	약 61 초	약 3.9 시간(13,993 초)

항목	표본(2 만 건)	전체(148 만여 건)
배송비 모델 학습 시간	약 60 초	약 3.9 시간(14,066 초)
데이터 나누는 방식	80%(학습) : 20%(검증)	동일

건수는 약 74 배 늘었는데, 학습 시간은 약 229 배나 늘었습니다. 단순히 비례해서 늘어난 게 아니라 훨씬 더 가파르게 늘어난 셈입니다. 나무를 하나씩 순서대로 심어가는 방식이다 보니, 데이터가 많아질수록 나무 한 그루를 심는 데 드는 시간도 함께 늘어나기 때문으로 보입니다. 그래서 여러 조건을 이것저것 바꿔가며 시험해 봐야 하는 작업은 시간이 짧게 걸리는 표본으로 먼저 해보고, 최종 확인만 시간이 오래 걸리는 전체 데이터로 하는 방식이 현실적으로 더 합리적입니다.

4.5 만들어 둔 함수를 그대로 재사용

오차를 계산하는 함수, 가격을 원래 단위로 되돌리는 함수, 나무의 가지 끝 개수를 세는 함수, 이 세 가지는 표본 분석 때 만들어 둔 것을 전체 데이터로 다시 돌릴 때도 코드를 전혀 고치지 않고 그대로 썼습니다. 처음부터 “데이터 양이 바뀌어도 그대로 동작하도록” 만들어 둔 덕분에, 이번에 별도로 손볼 부분이 없었습니다.

5. 결과 정리

5.1 표본 결과와 전체 데이터 결과, 얼마나 비슷했나

이번 1 차 분석에는 미리 정해 둔 목표 점수가 따로 없었습니다. 그래서 목표 달성 여부를 따지기보다는, 표본(2 만 건)으로 봤던 결과와 전체 데이터(148 만여 건)로 다시 본 결과가 얼마나 비슷한지를 중심으로 정리했습니다.

점수	표본(2 만 건)	전체(148 만여 건)	차이
RMSLE (가격, 낮을수록 좋음)	0.6558	0.6535	-0.0023
R ² (가격 설명력, 높을수록 좋음)	0.1055	0.0906	-0.0149
정확도 (배송비)	0.6723	0.6707	-0.0016
ROC-AUC (배송비 판별력)	0.7135	0.7123	-0.0012
Recall 평균 (배송비)	0.6535	0.6533	-0.0002
F1 평균 (배송비)	0.6524	0.6517	-0.0007

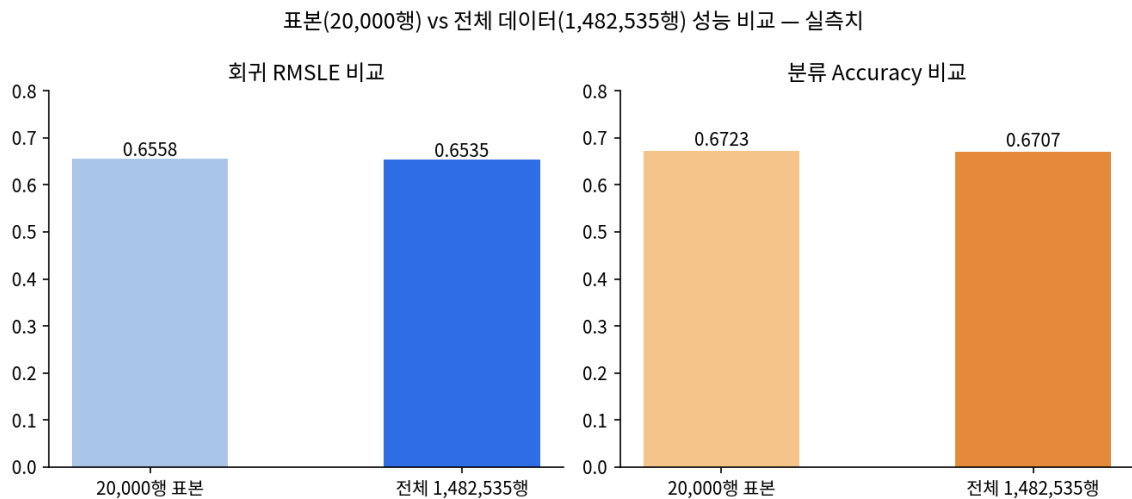


그림5-1. 표본(2 만 건) 과 전체 데이터(148 만여 건) 결과 비교

6 개 점수 모두 표본과 전체 데이터의 차이가 0.015 안쪽으로 매우 작았습니다. 이는 두 가지를 함께 말해줍니다. 첫째, 1 차 분석에서 뽑았던 2 만 건 표본이 전체 데이터의 특징을 왜곡 없이 잘 대표하고 있었다는 것이고, 둘째, 그렇기 때문에 앞으로 여러 설정을 바꿔가며 반복해서 실험해야 하는 작업은 4 시간 가까이 걸리는 전체 데이터 대신 1 분 남짓이면 끝나는 표본으로 먼저 빠르게 해보고, 최종 확정 단계에서만 전체 데이터로 다시 확인하는 방식이 실제로도 합리적이라는 근거입니다.

AIC·BIC 는 나무 100 개 조건에서만 전체 데이터로 다시 계산했습니다(AIC -250,652.0, BIC -242,161.5, BIC 가 AIC 보다 큰 관계는 그대로 유지). 나무 개수를 20/50/100/200 으로 바꿔가며 비교하는 실험은 4.3 절에서 밝힌 대로 계산 시간 문제로 아직 다 마치지 못해, 이번 결과만 가지고 “지금 설정이 통계적으로 과하게 복잡한지” 까지는 결론 내리지 않았습니다.

판매자부담 배송비를 잘 못 맞추는 문제(3.4 절, Recall 0.49)도 표본과 전체 데이터에서 똑같이 나타나, 표본이라서 생긴 우연이 아니라 데이터 자체에 있는 특징이라는 점이 다시 한번 확인됐습니다.

5.2 앞으로 더 살펴볼 것들

- 나무 개수(20/50/100/200)를 비교하는 실험과 관련 검증 — 계산 시간 문제로 아직 끝내지 못해, 다음 업데이트에서 이어서 진행할 예정
- 전체 데이터 학습 시간(약 3.9 시간)이 실무에서 쓰기엔 부담스러운 수준이라, 대용량 처리에 더 유리한 방식(HistGradientBoosting 계열)으로 바꾸면 시간이 얼마나 줄어드는지 확인
- 판매자부담 배송비를 잘 못 맞추는 문제를 개선할 수 있는지 — 표본·전체 데이터 모두에서 똑같이 관찰된 문제라 우선순위 있게 살펴볼 것
- 설정값을 자동으로 더 정교하게 찾아주는 방법(Hyperopt/Optuna 등) 적용 — 시간이 오래 걸리는 만큼, 표본으로 먼저 찾아본 뒤 전체 데이터로 최종 확인하는 2 단계 방식으로 진행